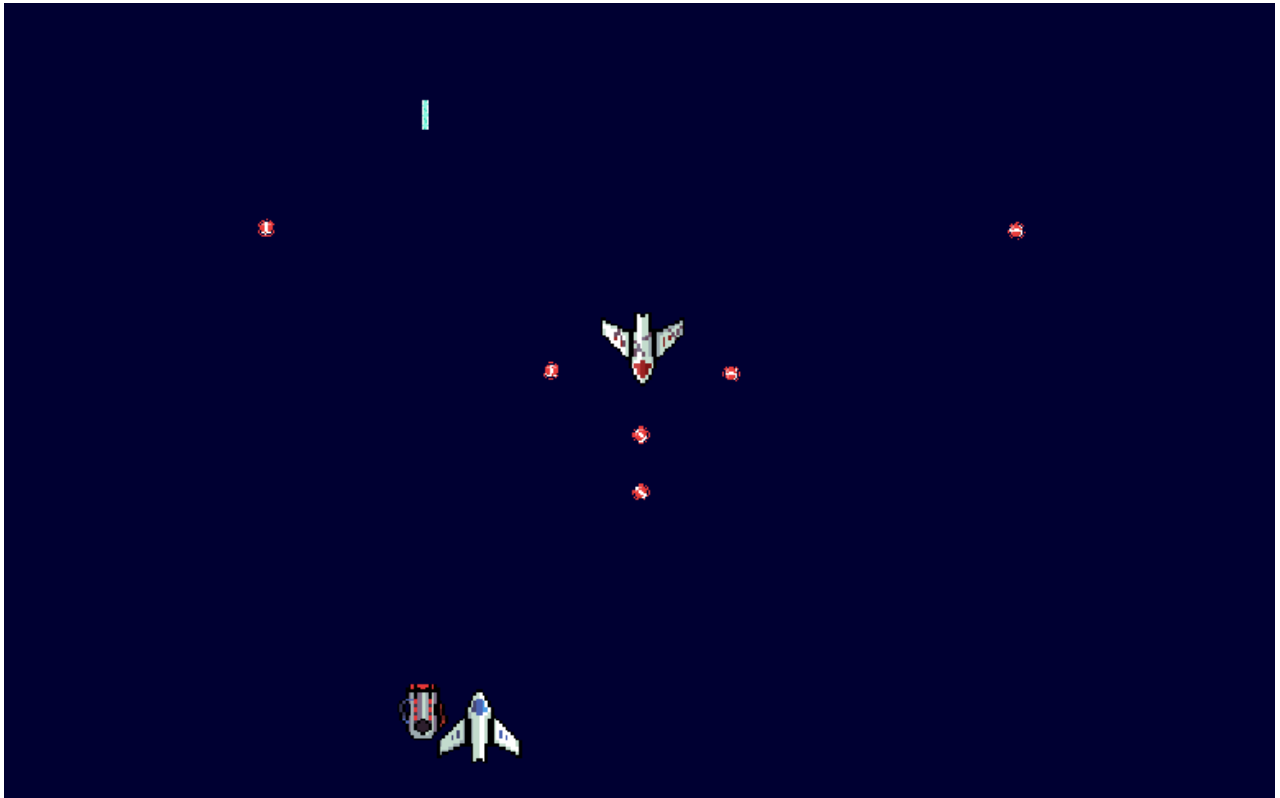


(Still) Untitled space invader game



Introduction

For our class in reusable conceptual models (MCR), we were asked to choose a design pattern and to apply it to a given context (in our case a game). We ended up choosing the builder design pattern and decided to make a bullet hell / shoot-em-up game inspired by the Touhou series and Bullet Hell Monday. We thought this type of game would fit in perfectly with the builder design pattern since it would allow us to easily build different types of enemies, weapons and even levels. We also wanted the player to be able to build and upgrade their ship.

Technologies

For the game's stack we were considering maybe doing it in unity but ultimately ended up choosing a LibGDX based stack for its supposed simplicity and because we are way more familiar with Java than C#. Unity also presented the challenge since it is a bit hard to pick up for the first time and 2 of our members have never used it.

Our project relies on the following technologies to run :

- **Java 17**: The core programming language
- **LibGDX** (as mentioned before): A really easy to pick up and use Java game development framework that provides features like OpenGL rendering, input handling, and asset management...

- **Gradle:** Tool used to build the project (comes with a default LibGDX project)

Game Features

The project currently integrates several key functionalities:

- **Home Screen & Configuration:** Provides an entry point for runs, allows for input toggling (keyboard or mouse), audio adjustments, and displays the team credits.
- **Economy, Progression & Points:** Features a shop system to exchange collected currency for ship improvements. A global scoring mechanic monitors gameplay and shows the total score at the end of a run.
- **Campaign Content:** Includes 4 unique stages with escalating difficulty.
- **Mechanics & Special Abilities:** Supports player movement and continuous fire, alongside powerful limited-use abilities like a barrage of beams or enemy destroying barrier.
- **Pattern Construction:** Utilizes our builder architecture to streamline the definition of new entities and levels. This also facilitates the addition of shield components to both hostile and player ships.

How to play

- **Navigation & Piloting:** Move your vessel in eight directions using the **WASD** or **Arrow Keys**. Alternatively, the settings menu allows for a switch to mouse-based navigation.
- **Combat Engagement:** Maintain sustained fire from all active weapon systems by holding the **Spacebar**.
- **Specialized Artillery:** Trigger your ship's unique high-impact ability—such as the beam or shield boost—by pressing **Q**, provided a charge is available.
- **Customization Phase:** Use the **Number Keys** during the shop interval to select new components. Finalize your current configuration and resume the campaign by pressing **Enter**.

Architecture

Diagram

See doc folder in our project. It doesn't fit here !

How does the game work (main loop)

In our game, everything runs within the **gameContext** class, it serves as our gameloop and it also manages our different assets and **GameObjects**.

GameObjects

GameObject is an abstract class that allows us to standardise how the game context will call them (for actions such as rendering, initialising and updating), It also allows us to manage the collisions between the game objects. By doing it this way, we ensure that we can easily add new classes to our game without having to do any changes to the game context's logic. Currently we have the following classes that extend from gameobject:

- **Ships (for both the player and the enemy)** Entities that move, take damage and equip Weapon and shield objects.
- **Shield** Provides extra hit points (HP) to a ship, both the amount of extra hp and how long it takes for the shield to recharge hp can be modified.
- **Weapon** manages spawning of projectiles, allows for them to be spawn only after a certain timeout, allows for there to be a sprite and offset if needed.
- **Projectile** represents the lasers fired from the ships, adapts the sprite to its trajectory and remembers who fired it (in order to try to optimise the number of collision calculations)
- **ScoreDrops(and its child classes)** Represents a drop that is dropped by defeated enemies and awards bonus points to the player when picked up.

Game States

The application is partitioned into several autonomous states, each implementing its own unique logic for the **render()** and **update()** cycles. We utilized the State design pattern to manage these transitions, facilitating the seamless addition of new phases and conditional switching. Our primary states include:

- **HomeScreenState**: Serves as the initial interface upon startup, presenting the main title and capturing player input. Selecting the play option triggers a transition to the **UpgradeMenuState**.
- **PlayingState**: Manages the active gameplay by spawning enemies according to the **CampaignSingleton** and iterating through all **GameObject** behaviors each frame. Upon failure, it shifts to **GameOverState**; upon campaign completion, it leads to **VictoryState**; otherwise, it directs the player to the **UpgradeMenuState** after each level.
- **UpgradeMenuState**: A dedicated interval for economy management, displaying current funds and allowing for ship component purchases. Pressing Enter resumes the campaign in the **PlayingState**.
- **GameOverState**: Displays the terminal "GAME OVER" screen and total points. The player can return to the **HomeScreenState** by pressing Space or Enter.

- **VictoryState**: Celebrates campaign success with a "VICTORY!" message and final scoring. Similar to the failure state, it waits for a keypress to return to the home screen.
- **OptionsState**: A configuration menu where users can adjust audio levels and switch between keyboard and mouse-based piloting.
- **CreditsState**: Displays the development team members and provides attribution to the LibGDX framework creators.

Builders

The selection of this specific game genre was primarily driven by the builder pattern, which we aimed to leverage extensively. Our implementation frequently couples these builders with directors—serving as our "cookbooks"—to provide streamlined templates for various system components.

Level Builder and Director

- **Structural Builder**: Facilitates the chaining of multiple **EnemySpawn** instances to define encounter timing. It enforces a non-empty spawn queue as a prerequisite for level initialization.
- **Global Director**: Houses the configurations for all campaign stages (**level1** through **level4**), centralizing stage logic within dedicated functions for easier expansion.

Enemy Builder and Director

- **Unit Builder**: Supports the fluid attachment of attributes such as **setSpeed()**, **setPath()**, and **addWeapon()**. It validates that sprites, trajectories, and firing parameters are present before object creation.
- **Hostile Director**: Acts as a repository for enemy archetypes, such as the **basicEnemy** and **heavyEnemy**, to simplify stage population.

Weapon Builder and Director

- **Ordnance Builder**: Handles the integration of multiple projectiles into a single system, allowing for custom spatial offsets, unique sprites, and specific velocities. It ensures every weapon is functional by requiring at least one projectile.
- **Weaponry Director**: Manages the primary weapon systems currently featured in the game:
 - **playerMainWeapon**: a simple straight shot.
 - **playerSideWeapons**: diagonal shot for broader coverage.
 - **enemySpreadWeapon**: A three-way pattern utilized by enemies

PlayerBuilder

Active during the **UpgradeMenuState**, this builder assembles the player vessel based on selected improvements. While multiple weapons can be combined, the architecture limits the ship to a single active shield.

ShieldDirector

Unlike other directors, this component functions without a builder; it serves to standardize shield generation and maintain architectural consistency throughout the project.

-

Conclusion

What we could have added

Although we are happy with how the game came out, unfortunately there are still a few missing features that we would have liked to implement if we had the time such as:

- Being able to add skins to the player, that the player could buy with money and keep across runs
- A (or many) dynamic background that would scroll with parallax behind each level.
- Bosses!!!!, a last challenge after each level with an healthbar, multiple phases and unique patterns
- A scoreboard or at least a way to score your best runs.
- Silly bonuses such as “eliminate every enemy in a level” or “beat a level without destroying a ship” would be cool ways to incentivise the player to play differently.